# PREDICTING EXECUTION PATHS AND BOUNDING OPTIMIZATION IN SPECULATIVE DECODING

**Anonymous Authors**[1]

## ABSTRACT

A reference mismatch rejects an execution path but does not predict its subsequent output. We test frozen complete-trajectory descriptions of speculative decoding, then examine their system consequences. On one B570 stack, a draft-free target-only UB1 predictor selected on exposed discovery matches 72/72 reserved capped ID sequences on 24 repeat-stable prompts, including four reference-divergent trajectories; the UB128 reference matches 60/72. This establishes predictive sufficiency for that path, without restoring qualification or identifying a unique cause. Separate same-state forks change scores but not sampled immediate decisions. A prospective RTX same-set comparison on all 19 remaining exposure-audited prompts rejects the specified target-only UB2 candidate (51/57), while proposal-conditioned replay matches 57/57 on shared numerical routines. UB2 and reference fail on different tasks; no necessity of history follows. On the same prompts, 171 fresh qualification/cost runs block batching and qualify serialization, which is slower (0.677× target/serialized latency) within an instrumented post-prefill boundary. Supporting evidence rules out draft/residual-only parity on 358H at fixed progress and target work, and shows qualified M4 Guard losing to fixed speculation. These finite results constrain both execution descriptions and optimization choices while keeping prediction, reference compatibility and complete cost distinct.

## 1 INTRODUCTION

Speculative decoding uses cheap proposals to amortize target work (Leviathan et al., 2023). On consumer systems, changing the execution path can change both cost and greedy output. Prior work already observes batch-shape sensitivity, applies correctness gates and measures slowdowns (Chordiya, 2026; Yuan et al., 2025; Zhang et al., 2026). A failed reference comparison therefore supplies an important deployment verdict, but leaves a different systems question unanswered: what execution description can predict the entire divergent path before it runs?

We study that question with frozen, complete generated-ID predictions. A predictor that only repeats the reference cannot explain a reference-divergent trajectory. A predictor sealed before live execution can instead be falsified at any later generated position. Its success constrains the execution description needed to reproduce the path; it does not repair the path or identify a unique numerical cause. We then ask which optimization choices remain possible under reference compatibility and complete realized cost. This connects diagnosis to implementation decisions without treating either accurate prediction or a cheap component as a speedup.

On one B570 Vulkan stack, exposed discovery selects a draft-free target-only UB1 predictor. Its complete predictions match 72/72 reserved capped sequences on 24 repeat-stable prompts, including four reference-divergent trajectories; the UB128 reference matches only 60/72. The added information is stronger than a mismatch count: this target-only program predicts every later ID without native draft proposals. It is an empirically sufficient description of the tested path, while the path remains blocked as a lossless replacement. A separate same-state fork study changes scores without changing sampled immediate decisions, limiting a stronger causal explanation.

RTX supplies a boundary to this result rather than a portability claim. A same-set prospective comparison on all 19 remaining exposure-audited prompts rejects the specified target-only UB2 candidate (51/57 arrays), while separately orchestrated proposal-conditioned replay matches 57/57 using shared numerical routines. UB2 and the UB1 reference have equal match totals but different failed prompts. Thus a micro-batch capacity label alone is insufficient for this candidate; the successful replay gives a reproducible richer description, without proving proposals or history necessary for every possible predictor.

The system consequence requires its own measurement. On those same RTX prompts, 171 fresh runs jointly test refer-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

ence equality and cost: batching is blocked, and qualified serialization is slower within the instrumented post-prefill boundary. Supporting studies delimit other optimization directions. On 358H, even zero draft and residual cost cannot reach parity if measured progress and target work are fixed. On M4, a qualified continuation guard loses to fixed speculation, while hindsight choice between the two measured fixed modes offers little aggregate opportunity. These are constraints on tested implementations, not hardware rankings or a universal performance law.

Our contributions follow this order: (1) prospective B570 full-trajectory prediction that succeeds on non-reference outputs, with a separate sampled intervention; (2) same-set RTX discrimination of predefined execution descriptions, followed by a matched qualification/cost decision; and (3) measured 358H accounting and M4 fixed-mode comparisons that identify which optimization directions the existing evidence can support. The contribution is finite predictive discrimination and its decision consequences, not a new decoder, a new numerical phenomenon or a successful adaptive policy.

## 2 Questions, Output Contract and Cost

We distinguish three questions: can an execution description predict a trajectory, does that trajectory preserve a specified reference, and does a qualified implementation reduce complete cost? Acceptance rate, isolated draft throughput and target-forward batching answer narrower questions.

### 2.1 Prediction and reference compatibility

Let $k$ be maximum draft width, UB the target micro-batch capacity ($n_{\text{ubatch}}$), and EOG a vocabulary end-of-generation ID. A greedy speculative round verifies proposals, commits the agreeing prefix and a correction or bonus when budget permits, then removes inconsistent suffix state. Our strict output contract compares complete committed generated IDs with the matched target-only greedy reference under the frozen model/runtime/configuration. A failed pair blocks a workload-global lossless optimization claim; a small passing probe establishes eligibility for a larger gate. Capped ID equality does not measure functional HumanEval correctness or pass@$k$.

Historical text-path agreement (H) compares arrays reconstructed from CLI text. Direct-ID qualification (Q) compares captured IDs with the study's reference. Prediction equality compares a sealed complete array with reserved native live IDs. Prediction can succeed on a reference-divergent path, so it does not imply Q. We write prediction equality in words; the letter P below denotes only the draft worker. Same-state intervention has separate score, immediate-decision and continuation endpoints.

### 2.2 Execution descriptions as testable candidates

Table 1 states what each program receives and computes. UB denotes capacity, not a trace of actual forward shapes, occupied positions or prior KV operations. In particular, target-only UB2 and native speculative UB2 need not issue the same sequence of forwards. The candidate test asks whether a specified program predicts complete output, rather than assuming matching parameter names imply matching numerical paths.

### 2.3 Complete realized cost

Let $T_b$ be matched target-only cost per output token, $T_d^{\text{real}}$ complete realized draft-subsystem time including interface replay, $T_{\text{target,round}}^{\text{real}}$ complete target work, and $T_x$ residual synchronization, transfer and bookkeeping outside those regions. Let $C$ count all committed output per round, including correction/bonus. Then

$$S = \frac{T_b C}{T_d^{\text{real}} + T_{\text{target,round}}^{\text{real}} + T_x} \tag{1}$$

$$= \frac{C}{D_{\text{real}} + R_{\text{target}} + X}. \tag{2}$$

The normalized denominator gives break-even $C > D_{\text{real}} + R_{\text{target}} + X$. These are accounting identities, not new profitability theory (Kim et al., 2026). Total-time ratios need not equal prompt geometric means. Incompatible experiments cannot isolate an unmeasured component by subtraction. The historical target-forward proxy $V_k^{\text{micro}}$ omits drafting and integration; its original cells and separate fresh qualification gates remain in Appendix F.

Primary results answer a frozen question after its applicable gate; secondary and post-study results retain their own scope. Path controls or measured regions support a bounded explanation, while timing on blocked paths remains diagnostic. A later Q gate does not retrospectively certify historical H timing.

## 3 Study Design and Evidence Scope

### 3.1 Frozen systems and complete endpoints

Each study has its own protocol, tasks and comparator, stated beside its results and mapped in Appendix R. The target/draft pair is Qwen3-8B/0.6B Q8_0, except the 358H adapter's symmetric INT4 OpenVINO draft. Final comparisons process one request at a time. Historical settings are in Appendix K and RTX V4 in Appendix Q; hardware, backend, OS and quantization jointly vary, preventing hardware-only causal comparisons. RTX's original direct-ID OS build is 26200; follow-ups use 26300. A driver version alone does

*Table 1.* Inputs and local execution contracts. Each program starts from its own prompt prefill. P = draft worker; R = target worker. Shared numerical routines limit implementation independence. UB labels do not specify identical actual forward traces.

| Program | Generation inputs | Locally computed history / decisions | Separation or missing equivalence |
|---|---|---|---|
| B570 H1 | Prompt; target weights; UB1; no draft or proposals | Own target-only generated prefix, KV and greedy decisions | Draft-free predictor; no unique-cause isolation |
| RTX target-only UB2 | Prompt; target weights; UB2; no draft or proposals | Own target-only prefix and KV; one generated-ID stream | Capacity-level candidate; equality of actual native forward traces is not established |
| RTX H2 | Prompt; P's proposals from its own earlier R feedback | R checks own prefix/pending ID; computes sampling, accept/correct/bonus and KV rollback | Separate P/R orchestration; shared llama.cpp/GGML/CUDA |
| RTX fresh R_ONLY | Prompt; sealed H2 proposal requests, read sequentially | Fresh R computes its own target decisions and state transitions | Target-only execution of tape; tape generation uses draft; same numerical routines |
| Native RTX live | Prompt; its own draft proposals; no tape import | Native target/draft history, acceptance and state transitions | Independent live outcome; not an independent numerical implementation |

not identify driver bytes.

"Complete" means every committed/generated ID through the first inclusive EOG or 128-ID cap, excluding prompt IDs and uncommitted proposals. B570 H1 reaches the cap in every reserved run; RTX also has earlier EOGs. Exposure audit and deterministic selection precede reserved execution, and predictions/tapes are sealed before native live. V4 selects all 19 tasks remaining after 145 exposure exclusions, including seven exposed by failed V3. A freezes four sets of 57 pre-live arrays, then executes 57 native live runs; B measures 171 fresh runs on the same tasks. These are 19 distinct prompts, not 456 independent samples or two independent cohorts. No-known-exposure covers audited records only. Appendix Q retains the original failed V3 status, host-policy amendment and endpoint rules.

Historical M4 includes a representative 24-prompt SpecBench sample (Xia et al., 2024) and a separate preregistered 30-task HumanEval study (Chen et al., 2021); they are background, not replications of the later runtime study. The latter retains its own calibration, qualification and held-out splits in Section 6. All studies keep their original stop rules and comparators.

### 3.2 Cost estimators and conditional uncertainty

Later RTX cost studies take each condition's median of three times per prompt, then reference/candidate ratios and their prompt GM. V4 uses 10,000 paired-row bootstrap draws, seed 2026100204, frozen task order and linear .025/.975 percentiles. This resamples the empirical distribution of the selected prompt rows, conditional on the exposure-audited cohort, observed condition medians and frozen configuration. It does not account for exposure selection, predict a HumanEval population failure rate or cover a new thermal/session distribution. Repetitions are not independent bootstrap units. Cost interpretation requires every reference-ID pair and stable reference repeats; blocked timing remains diagnostic.

The 358H prompt estimator takes the median of three paired ratios before the prompt GM; total-time accounting is separate. M4 runtime uses condition medians and its own frozen random streams. Historical H estimators remain in Appendix B. All intervals omit full session and recalibration uncertainty.

Timing excludes loading and initial prefill. The RTX V4 timer begins after synchronized target/draft prefill and before per-request sampler/state work, ending at slot release. The M4 and 358H boundaries differ and are stated with their results. M4 package energy excludes separate DRAM, B570 uses adapter-board energy, and RTX mixed domains do not form a common system boundary. There is no end-to-end 358H Phase-3 or runtime energy result.

### 3.3 What offline evidence can establish

The compact anonymous index directly recomputes retained arrays, captured timer intervals and statistical summaries. Full capsules additionally retain original logs, proposals, process ledgers and code; projected state-check booleans are not raw KV or opaque sampler bytes. Source/hash mappings connect derivatives with these retained records. Neither package authenticates omitted weights/runtime/compiler/driver bytes or original absolute process time. Hashes identify content; host records and server acknowledgements support reported ordering. Appendix O gives exact entries and Appendix Q the V4 capabilities. Offline verification executes no models.

**Existing-record analysis.** Score/margin descriptors, fixed-mode opportunity and conditional cost bounds are post-study analyses with aggregate outcomes already known. The v0.8.8 plan similarly fixes the existing 171 V4 times, prompt grouping and linear-quartile summaries before the new calculation. Absolute latency describes 19 prompt medians per condition; it neither replaces the paired GM/CI nor creates a speedup for blocked batching. Appendix P retains earlier analyses and Q the V4 descriptors.

## 4 Prospective Execution-Path Prediction

### 4.1 B570: a draft-free full-trajectory predictor

A target-only harness first reproduces 90/90 UB128 reference sequences. On 12 exposed discovery tasks, H1 target-only UB1 matches 36/36 native serialized trajectories. The frozen decision tree selects H1 and leaves H2 unrun. Conservative exposure exclusion then selects 24 reserved tasks. H1 freezes before reserved reference/predictor inference, and all 72 complete predictions freeze before reserved native live (Figure 1).

Prediction/live agrees 72/72 over three stable runs per prompt, including all 12 runs of four reference-divergent prompts. The same-set UB128 reference agrees only 60/72 and remains *BLOCKED*. Every reserved array reaches 128 generated IDs. Any wrong later ID would falsify the prediction, rather than merely reproduce a first-difference signature. The four first differences occur on /134 at index88 (914→1946), /42 at 73 (432→1817), /145 at 103 (1184→686), and /138 at 93 (14374→9064); all later IDs are also predicted. Appendix J retains every committed array and freeze role.

This answers a question that qualification alone cannot. Reference mismatch establishes that the path is unsuitable for strict replacement; it does not establish whether a target-only program can describe its full output. Here the selected target-only UB1 program suffices without receiving draft weights, proposals or future live IDs. That result rules out the need for those inputs *in this successful predictive description*. It does not show that drafting is causally irrelevant inside native execution, that UB1 is the unique predictor, or that the output contract is restored.

### 4.2 Sampled numerical sensitivity is not an immediate split

A separate frozen fork study uses 24 new prompts $\times$ three, sampling the first verification block after at least 64 committed IDs. UB1/UB128 branches share checked KV, prefix, positions, proposals and cloned sampler state. Scores change at 72/72 forks, but raw top-one and actual immediate decisions change at 0/72. The immediate comparison contains 195 reachable paired decisions; 216 raw-logit rows are correlated within prompts and blocks.

Existing-record descriptors put these results on a score scale. Median absolute top-one score change is 0.0695, with p95 0.3437 and maximum 2.3503. Raw top-two margins have minima 0.2291 (UB1) and 0.2003 (UB128), and medians 17.5021 and 17.5128. These are marginal descriptors, not a certified decision-boundary distance or a uniform perturbation bound. In particular, the study does not supply a sampled near-tie witness. Changed scores with stable sampled decisions show local numerical sensitivity and immediate stability, not a whole-generation null effect.

The alternate branch restores draft state and continues with UB128 target blocks; later proposals follow its altered live history. Complete trajectories differ in 18/72 runs on 6/24 prompts, first diverging 10–50 positions after the fork. Persistent continuation therefore confounds attributing a delayed difference to one sampled block. The study neither explains the original H1 four tasks nor establishes immediate reference restoration. Appendices L/P retain score/margin distributions, all six later-sensitive prompts and clone/R0 controls.

### 4.3 RTX: discriminating predefined candidates on one set

Earlier RTX H1 fails exposed discovery at 24/36 and stops with zero reserved runs. A separate V2 study obtains 72/72 matches from proposal-conditioned H2 and fresh R_ONLY but lacks a same-set target-only competitor (Appendix M). V4 addresses that missing comparison; it is not a retest of one portable B570 predictor.

All 19 remaining exposure-audited tasks enter V4. Four pre-live conditions—UB1 reference, target-only UB2, H2 and fresh R_ONLY—produce 228 complete arrays. All arrays/tapes are sealed and remotely retrieved before 57 native batched live runs. H2's draft worker P uses its

| Stage | Design or comparator | Complete generated-ID endpoint |
|---|---|---|
| Discovery | 12 prompts × 3 runs; select target-only UB1 as H1. | UB1 predicts the serialized trajectory in **36/36** runs. |
| Hypothesis freeze | After discovery; fix H1, reference, tasks and endpoint. | Before reserved reference/predictor inference; H2 remains unrun. |
| Prediction freeze | After pre-live inference; seal all 72 UB1 arrays. | Before the first reserved live outcome; no prediction edits. |
| Live test | Compare each of 72 live serialized runs with both frozen paths. | UB1 predictor/live **72/72**; UB128 reference/live **60/72 (BLOCKED)**. Four prompts (12 runs) are reference-divergent. |
| Divergent case | HumanEval/134; first difference at generated-ID index 88. | Reference ID 914 → predictor/live ID 1946 in all 3 runs. |

*Figure 1.* **B570 prospective prediction.** Hypothesis and full-array freezes are separate stages. The 24 prompts × three repeat-stable runs test complete capped trajectories, including four non-reference predictions. Reported ordering and retained arrays are reviewed in the appendix; prediction success does not restore qualification.

*Table 2.* RTX V4 complete prediction/live agreement on 19 prompts × three. Separate P/R orchestration uses shared numerical routines. Below: full-array failures on all distinguishing tasks, stable in three repeats; replay matches every task. These are prediction comparisons, not qualification.

| Predictor / comparator | Pairs / 57 | Prompts / 19 |
|---|---|---|
| Target-only UB2 | 51 | 17 |
| Separate P/R (H2) | 57 | 19 |
| Fresh R_ONLY | 57 | 19 |
| UB1 reference | 51 | 17 |

| Task | UB2 | Reference | H2 / R_ONLY |
|---|---|---|---|
| /15 | Fail | Fail | Match / Match |
| /18 | Match | Fail | Match / Match |
| /25 | Fail | Match | Match / Match |
| Other 16 | Match | Match | Match / Match |

own earlier target feedback to construct proposals; target worker R computes its own decisions. Fresh R_ONLY reads the sealed tape sequentially, and native live imports no tape. This separates application/API orchestration on shared llama.cpp/GGML/CUDA numerical routines (Table 1), not adversarial OS access or an independent numerical implementation.

Table 2 shows complete-array agreement and the cross comparison hidden by aggregate totals. Target-only UB2/live is 51/57 on 17/19 prompts. H2/live, R_ONLY/live and H2/R_ONLY are each 57/57; both programs predict the two reference-divergent trajectories. All five conditions are repeat-stable. UB2 and reference both score 51/57, but /18 refutes only the reference and /25 only UB2; /15 refutes both.

The prospective result rejects the specified UB2 program's full-cohort sufficiency. Matching capacity does not establish identical occupied forward shapes, inputs or prior state transitions. Replay supplies a successful richer execution description; its success does not reject every target-only hypothesis or prove proposal/history necessary, minimal or unique. Tape construction uses drafting, and replay shares numerical code. Appendix Q gives first-difference IDs, complete endpoints and the failed V3 capture history; V3 contributes no observations to these denominators.

## 5 Qualification and Optimization Limits

### 5.1 RTX: matched qualification and complete cost

V4 B uses A's 19 tasks but 171 fresh observations: target-only UB1, batched $k = 2$ UB2 and serialized $k = 2$ UB1. One frozen timer executable, model/runtime, input, endpoint and balanced order schedule cover all three paths; each condition occupies each order position 19 times. The global native micro-batch option also propagates to draft, making this a whole-path configuration comparison, not an isolated target intervention.

The fresh ID gate blocks batching at 51/57. Serialization qualifies at 57/57, but its prompt-median target/candidate GM is 0.677× [0.662, 0.692], with 0/19 prompts faster (Table 3). Reciprocal relative GM latency is 1.476×. The experiment excludes the measured serialized configuration as a latency optimization and establishes no fast qualified alternative. Blocked batching supplies no admissible speedup or CI.

The new absolute latency descriptors show the scale behind the ratios (Table 4). Median across the 19 prompt medians is 1.607s for target and 2.332s for serialization; the full ranges are 0.532–2.920s and 0.843–4.102s. These post-study distributions do not estimate the paired GM by dividing their medians. Appendix Q retains every prompt-condition median and the unchanged primary estimator. The earlier two-path 30-prompt result, 0.646× [0.630, 0.660], uses a different cohort and has no batched condition; the two estimates do not form an optimization trend.

*Table 3.* RTX V4 matched qualification and cost: 171 fresh runs on 19 prompts × three × three paths. Target/serialized use UB1, batching UB2; speculation uses $k = 2$. Speedup = target/candidate latency. Only the blocked batched row has null speedup/CI. Serialization is faster on 0/19 prompts.

| Path | ID gate | Speedup [95% CI] |
|---|---|---|
| Target | Reference | 1.000 |
| Batched | 51/57 blocked | — |
| Serialized | 57/57 qualified | 0.677 [0.662, 0.692] |

*Table 4.* RTX V4 absolute prompt-median post-prefill latency (s): 19 medians per condition, each from three runs. Post-study spread, not uncertainty. Batched times are diagnostic; these distribution medians do not estimate the paired speedup.

| Condition | Min | Q25 | Median | Q75 | Max |
|---|---|---|---|---|---|
| Target | 0.532 | 1.441 | 1.607 | 2.396 | 2.920 |
| Batched | 0.641 | 1.730 | 1.910 | 2.953 | 3.248 |
| Serialized | 0.843 | 2.074 | 2.332 | 3.611 | 4.102 |

The native timer charges per-request sampler/state work after synchronized prefill, drafting, target validation, acceptance/correction/bonus, KV rollback, synchronization and related logging/response through slot release. Loading, prefill and teardown are excluded. Observer controls check output stability, while clock probes bound clock-call overhead only; condition-dependent debug logging need not cancel. Thus the conclusion concerns instrumented post-prefill request latency, not logging-free deployment, throughput or energy. The bootstrap conditions on the selected rows and observed medians as stated in Section 3.

### 5.2 358H: a bound on draft-only optimization

The closed 358H Phase-3 runner directly qualifies CPU and NPU drafting at 90/90 pairs per path, yet both are slower than target-only: CPU 0.734× [0.693, 0.771], NPU 0.352× [0.333, 0.369], with zero of 30 prompts faster. Isolated NPU drafting is faster than CPU at tested contexts, but the integrated synchronous OpenVINO region combines full-prefix replay/reconstruction and proposal. Its 58.35% share cannot be separated into the two costs (Figure 2). This is an interface result, not a general NPU limitation.

Measured committed progress is $C = 2.4631$ CPU and $2.4584$ NPU, including correction/bonus. Complete target work consumes 87.34% of CPU-path time; residual orchestration is 0.04% CPU and 0.02% NPU. Processed-total accounting gives $(D_{\text{real}}, R_{\text{target}}, X) = (0.4304, 2.9767, 0.00124)$ CPU and $(4.1251, 2.9435, 0.00130)$ NPU. Their 0.7227× and 0.3477× total-time ratios differ from prompt GMs.

At fixed measured progress, target work and baseline totals, nonnegative draft/residual cost implies $S \le C/R_{\text{target}}$: 0.8275× CPU and 0.8352× NPU, even if both other terms vanish. This adds a constraint beyond observing slowdown: improving those terms alone cannot reach parity under the stated accounting. The bound is post-study arithmetic, not a prompt-GM bound or measured optimization; changing progress or target execution invalidates it. Appendix P gives the exact totals. A persistent-state/wider-target candidate stops at qualification, so it supplies no formal performance or energy result.

CPU restore/reconcile correctness could not be established under the tested state representation; the NPU state-capable path was unavailable with the frozen dynamic-shape IR. Separate correctness-only controls reproduce UB1/UB2 greedy divergence on the target even with exact matching prefixes, and proposal replay reproduces speculative divergence without live NPU draft inference. These controls identify no unique floating-point operation. The implementation motivated by cost remains blocked by output compatibility; neither device is proved incapable of retaining state.

## 6 Runtime Control under Qualified Fixed Modes

The separate M4 runtime study tests whether continuation control improves a qualified speculative path. It uses its own frozen policy, direct-ID gate and cost boundary; historical M4 timing is background rather than its qualification evidence.

### 6.1 A prospective cost guard on M4

The M4 experiment asks whether a frozen rule based on already-realized complete cost can control continuation while preserving exact output and profitable execution against target-only. It does not assume that adding control improves on fixed speculation. We retain the Qwen3-8B/0.6B Q8_0 pair on llama.cpp Metal, fixed $k = 2$, greedy generation, and a 128-token cap. Guard begins on the same speculative path as Always-Spec and can fall back permanently to target-only within a request. It changes neither proposal width nor device placement.

At each eligible completed round, Guard divides all elapsed charged generation cost by committed output progress $C$. This count includes the initial target seed and committed correction or bonus tokens, but excludes rejected proposals. Guard continues if this cumulative cost per token is at most $(1 - m)R_{\text{ref}}$, a target-only reference frozen from calibration; otherwise it falls back. If a decision is reached with zero committed progress, Guard also falls back. Before each control decision, invalid candidate suffix state is removed from the target and draft contexts, preserving

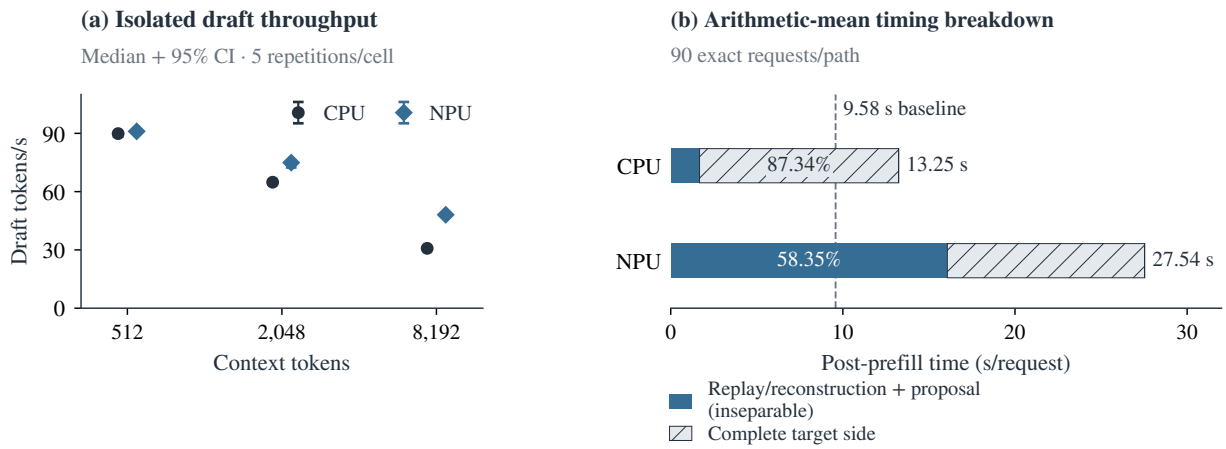



*Figure 2.* **Isolated draft advantage and complete 358H cost.** A: five-repeat steady-state draft throughput with repetition-bootstrap intervals. B: arithmetic mean post-prefill seconds over 90 exact observations per path, against the shared baseline. NPU's 58.35% region combines full-prefix replay/reconstruction and proposal; it is inseparable. CPU complete target work is 87.34%. These bars differ from the prompt-GM speedups in Section 5.2; persistence savings are unmeasured.

committed history for target-only continuation. The enclosing post-prefill clock includes generation-loop initialization, seed-token sampling, drafting, target work, synchronization, state reconciliation, control, and transition costs. Model loading, sampler/speculator setup, initial prompt prefill, and the speculative begin hook precede the timer; the result is not whole-request latency.

Calibration freezes $R_{\text{ref}}$ as the median of prompt-median target latency per output token. A preregistered grid selects decision round and margin by exactness and calibration GM latency relative to the two-fixed-mode oracle, with frozen tie-breaks. The selected rule has margin 0, first decides after 8 rounds, then after each later completed round. Early-ending requests make no decision. Online control receives no task/family labels, future length, request-specific target latency or oracle outcomes.

### 6.2 Held-out evaluation

Historical outcomes informed the design; prospective means policy freeze before reserved outcomes. Exposure exclusion and deterministic selection give disjoint calibration, qualification and held-out splits of 40, 16 and 60 prompts, balanced across HumanEval and Spec-Bench. Policy and reference precede qualification/formal measurement; the appendix retains the audited freeze-hash metadata defect. Always-Spec and Guard each qualify 48/48 direct-ID pairs.

The prospective M4 study runs 3 repetitions of target-only, Always-Spec, and Guard on each held-out prompt, randomizing condition order within prompt and repetition: 540 observations. Both speculative conditions preserve 180/180 complete direct-ID sequences, with zero mismatches. For each prompt and condition, analysis takes the median latency across repetitions, then the geometric mean of prompt latency ratios. This ratio of condition medians differs from the historical M4 text-path hardware estimator. Confidence intervals use 10,000 prompt bootstrap resamples with seed 20260924; repetitions are not independent sampling units.

*Table 5.* M4 ratios of paired prompt medians. Both speculative policies preserve 180/180 direct-ID sequences. Speedup and relative latency have opposite favorable directions. The oracle selects the lower of two measured fixed-mode medians with hindsight. Guard/Always is post-study secondary; values above one mean Guard is slower.

| Ratio (×) | GM | 95% prompt CI |
|---|---|---|
| *Speedup: above one favors speculation* | | |
| Target / Guard | 1.3114 | [1.2487, 1.3787] |
| Target / Always-Spec | 1.3469 | [1.2867, 1.4088] |
| *Relative latency: lower is better* | | |
| Guard / offline oracle | 1.0278 | [1.0137, 1.0446] |
| *Post-study paired secondary latency ratio* | | |
| Guard / Always-Spec | 1.0271 | [1.0130, 1.0436] |

Guard is faster than target-only within the charged-loop boundary on this held-out workload (Table 5). Fixed speculation has the higher aggregate speedup point estimate. A reviewer-requested post-study paired secondary analysis of the frozen prompt medians gives Guard/Always-Spec la-

tency 1.0271 [1.0130, 1.0436]. The interval is above parity: Guard had higher latency on this comparison. By prompt median, Guard is faster on 24 prompts, Always-Spec on 36, with 0 ties. Guard falls back in 30.6% of its 180 request executions; this measures control activity, not the fraction of beneficial decisions. Guard's geometric-mean latency relative to the offline oracle is 1.0278×. That oracle chooses the lower target-only or Always-Spec median after measurement and was unavailable to the runtime. It is a descriptive hindsight comparator, not an achievable-policy guarantee.

A post-study reconstruction of all original medians finds target-only faster than Always on only 2/60 prompts, both Spec-Bench. Always/oracle is 1.0007×: little opportunity for choosing between these two measured fixed modes. Guard falls back in all six executions of those two prompts but still loses to target-only on their medians. Guard also remains slower in a family-stratified paired bootstrap sensitivity analysis; Appendix P reports this separate new interval. These are hindsight comparisons, not transition-cost estimates.

Guard preserves IDs but supplies no incremental benefit over Always-Spec. Fallback activity does not establish beneficial decisions or risk reduction. No runtime energy claim is made. Policy transfer and stable small-stratum effects remain unestablished; the archived policy-freeze defect and original stopped pipeline markers are documented separately.

## 7 System Implications and Research Outlook

The studies constrain different choices (Table 6); they do not identify one cause or rank stacks. Qualification already rejects a reference-divergent implementation. The extra scientific information is whether a predefined execution description can predict that path before it runs, including every ID after divergence. B570 succeeds without proposals in its predictor; RTX rejects its specified capacity-level candidate on the same prompts where replay succeeds. Neither result licenses native replacement or proves a minimal state representation.

**Execution descriptions should be explicit.** These results motivate treating effective numerical execution as part of an experimental output contract. A micro-batch setting is a candidate description, not a guarantee of equal forward histories. The successful B570 description and failed RTX one tell an investigator which already-tested simplification survives, rather than supplying a universal kernel explanation. Replay can serve as a reproducible description when it succeeds, but shared kernels limit its independence. Stronger causal attribution would require controlling subsequent policy and checking relevant state, beyond complete-array agreement.

**Optimization targets should follow complete cost.** For RTX, legality and latency are measured together: the apparent alternative of serialization satisfies IDs but does not improve measured cost. For 358H, reducing interface replay alone cannot overturn the fixed-progress target-work bound. A design that aims for parity must change progress, target execution or the reference/baseline assumptions and qualify that changed path anew. Isolated NPU speed and target-forward proxies cannot establish that such a change works.

**Control opportunity precedes policy complexity.** On M4, fixed speculation is already close to the hindsight comparator over the two measured modes. Guard's fallback activity does not show that a large recoverable loss exists. This favors assessing the available fixed-mode opportunity before attributing benefit to continuation control. It does not bound all possible within-request policies: a better policy could alter decisions and transition costs, but that gain is not in the current measurements.

These are design constraints and research directions supported by finite studies, not measured improvements. Their reusable element is the separation of predictive description, reference compatibility and complete-cost opportunity; each concrete conclusion remains conditional on the tested path, workload and measurement boundary.

## 8 Related Work

**Performance and heterogeneous execution.** Speculative decoding amortizes target computation under its algorithmic assumptions (Leviathan et al., 2023). Prior work already documents verification and workload limits: Liu et al. (2026b) study verification dominance and batch effects in vLLM; Kim et al. (2026) account for conditional execution cost and runtime control; Lyu et al. (2026) derive edge–cloud profitability conditions. On consumer Apple silicon, Chordiya (2026) report gains and slowdowns across five backend configurations, diagnose ineffective batch-parallel verification in one quantized Metal path, test output agreement, and observe target-only batch-shape argmax changes. Slowdown and correctness gating have already been studied, as has heterogeneous placement: Dovetail, DuoDecoding, and Mirror-SD study GPU/CPU or GPU/NPU paths (Zhang et al., 2025; Lv et al., 2025; Bhendawade et al., 2025); Ruiz y Mesa et al. (2026) combine an analytical cost model with compiler-oriented CPU/GPU partitioning, and Lever jointly optimizes draft, verification, and CPU/NPU execution for a smartphone target (Wang et al., 2026). State reuse has prior art in compressed draft KV memory (Yuan et al., 2026). Our 358H study measures one interface that combines replay and proposal; it establishes no general placement or persistence result.

*Table 6.* What each study adds to an engineering verdict. Decisions are interpretations of retained evidence, conditional on the last column; no proposed change is a measured optimization.

| Study | Observation beyond a basic gate | Allowed investigation / implementation decision | Fixed premise or unresolved scope |
|---|---|---|---|
| B570 | Full non-reference trajectories predicted without proposals | Retain UB1 target-only as an empirically sufficient description; still reject live as strict replacement | One stack; predictive inputs are not native causal irrelevance |
| RTX A | UB2 and reference fail on different tasks; replay succeeds | Reject this predefined UB2 description; retain richer replay as successful | No universal necessity/minimality; shared numerical routines |
| RTX B | Qualified serialization loses with matched cost | Exclude measured serialization as latency optimization; no qualified batched claim | Instrumented post-prefill boundary; same selected prompts |
| 358H | Zero draft/residual bound remains below parity | Draft/residual-only savings are insufficient; parity needs a changed premise | Progress, target work and baseline totals fixed |
| M4 | Always near two-mode oracle; Guard slower | Fixed speculation remains better aggregate measured choice; activity is not benefit | Hindsight two-mode comparison; no bound on arbitrary policies |

**Numerical paths and output contracts.** Table 7 separates the closest questions. Numerical sensitivity, deterministic inference construction, correctness forensics and cost gating are established prior work. Our central addition is a frozen test of complete native trajectories that may differ from the reference, followed by a same-set candidate discriminator and matched qualification/cost. A separate sampled intervention constrains causal interpretation. We claim no first observation of numerical sensitivity, independent numerical implementation or unique kernel cause.

**Adaptive speculation and the study boundary.** TurboSpec adjusts speculation for serving goodput (Liu et al., 2025); BanditSpec selects configurations online (Hou et al., 2025); ACL HeteroSpec adapts verification from contextual entropy (Liu et al., 2026a). Adaptive control is established prior work. Our M4 Guard is a qualified runtime control test, not a new policy advantage: it beats target-only, but fixed speculation is faster in the post-study paired secondary comparison. Our measurements test prospective direct-ID qualification, frozen trajectory prediction, sampled intervention effects and realized cost on qualified paths.

## 9 Limitations

Narrow Qwen3 models, greedy caps and jointly varying hardware/backend/OS/quantization limit inference: no hardware ranking, population failure rate, functional pass@$k$ or production claim follows. Strict-ID compatibility is reference-conditional; historical text-path timing remains distinct. RTX V4 compares all three paths within one cost study, but batching is blocked. Its timer includes logging and supplies no qualified batched speedup, deployment estimate, throughput or energy result.

B570 H1 has 24 reserved prompts and four reference-divergent trajectories on one stack. The fork samples one checkpoint per new run; persistent UB128 continuation is not a single-block restoration or an explanation of the old four tasks. RTX V4's 19 prompts are all remaining exposure-audited tasks, not a representative population or an independent B cost cohort. Its UB2 failure rejects only that candidate; matching capacity does not establish actual forward-trace equivalence. Replay uses drafting and shared numerical routines; necessity, minimality and uniqueness of proposal/history remain unknown. Native micro-batch settings also affect draft, so B compares whole-path configurations. No unique operation or portable numerical cause is identified.

Intervals resample selected empirical prompt rows conditional on observed medians and configuration; they exclude exposure selection and session/recalibration variation. Events/repeats are correlated. Exposure checks cover retained records. Hashes identify bytes; reported host/server order, Git and anonymous ordinals do not authenticate original absolute time or omitted binaries. 358H runtime raw traces and exact old sequence ZIP are unavailable; preserved arrays and processed cost records remain independently checkable. Its timing region combines reconstruction/replay with proposal, so their separate costs are unavailable. The zero-draft bound holds only at fixed measured progress and target work.

*Table 7.* Closest prior questions and the bounded question here. This is a scope comparison, not an exhaustive novelty or priority claim.

| Work | Established question / result | Distinction of this study |
|---|---|---|
| Chordiya (2026) | Consumer-backend gains/slowdowns, correctness gates and target-only batch-shape argmax changes | Seal complete non-reference predictions before native live; discriminate predefined candidates on one set |
| Yuan et al. (2025) | Numerical output variation across system configurations, including batch size and precision | Test a specific program's prospective full-trajectory sufficiency, rather than only observe variation |
| Gond et al. (2026) | Construct deterministic verified speculation with shape-consistent verification and KV repair | Predict an existing native path that may remain reference-incompatible; do not construct a deterministic decoder |
| Zhang et al. (2026) | Distinguish ragged-batch synchronization errors from residual numerical mismatch, with correctness/cost analysis | Use prospective path-description comparisons and an own-reference matched three-path cost study; no new synchronization invariant |

M4 Guard/Always, family sensitivity and fixed-mode opportunity are post-study secondary. The oracle chooses two measured modes with hindsight, not arbitrary switching policies. The M4 metadata defect retains original error and stopped markers. Runtime energy and 358H persistence are unmeasured; energy domains differ; M4's corrected synthetic ceiling was unexecuted.

## 10 Conclusion

Frozen B570 target-only predictions match 72/72 reserved capped trajectories, including four non-reference prompts; the reference matches 60/72 and remains blocked. A separate intervention changes scores but not sampled immediate decisions. RTX discriminates predefined descriptions: target-only UB2 matches 51/57, replay 57/57, with distinct UB2/reference failures. These results constrain predictive descriptions without identifying necessary history or a unique numerical cause. Matched RTX cost then rejects instrumented serialization as an optimization; 358H requires changing the fixed progress/target-work premise to reach parity through cost savings; M4 Guard loses to fixed speculation. Prediction, reference compatibility and complete-cost opportunity provide different information, and each must be tested on its own stated path.

## REFERENCES


Bhendawade, N., Nishu, K., Kundu, A., Bartels, C., Cho, M., and Belousova, I. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161v2*, 2025. URL https://arxiv.org/abs/2510.13161v2.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021. URL https://arxiv.org/abs/2107.03374v2.

Chordiya, P. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026. doi: 10.48550/arXiv.2607.17283. URL https://arxiv.org/abs/2607.17283.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026. URL https://arxiv.org/abs/2601.17768v2.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 24045–24079. PMLR, 2025. URL https://proceedings.mlr.press/v267/hou25h.html.

Kim, S., Kim, J., Cho, Y., and Seo, J. Runtime-controlled evaluation of speculative decoding methods. *Electronics*, 15(14):3179, 2026. doi: 10.3390/electronics15143179. URL https://www.mdpi.com/2079-9292/15/14/3179.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, S., Ye, Y., Zhu, Q., Cao, Z., and He, Y. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12930–12947. Association for Computational Linguistics, July 2026a. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066v3*, 2025. URL https://arxiv.org/abs/2406.14066v3.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026b. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025. URL https://arxiv.org/abs/2503.00784.

Lyu, Y., Irukulapati, B., and Champati, J. P. Speculation at a distance: Where edge-cloud speculative decoding actually pays off. *arXiv preprint arXiv:2606.25091v2*, 2026. URL https://arxiv.org/abs/2606.25091v2.

Ruiz y Mesa, A., Korol, G., Riesterer, M., Cardoso de Lima, J. P., and Castrillon, J. Compiler-assisted speculative sampling for accelerated LLM inference on heterogeneous edge devices. In *8th Workshop on Accelerated Machine Learning (AccML), co-located with HiPEAC*, January 2026. URL https://accml.dcs.gla.ac.uk/papers/2026/8th_AccML_paper_8.pdf.

Wang, T., Li, F., Sun, Y., Gao, W., and Ren, J. Lever: Speculative LLM inference on smartphones. *arXiv preprint arXiv:2605.16786*, 2026. doi: 10.48550/arXiv.2605.16786. URL https://arxiv.org/abs/2605.16786.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https:

//aclanthology.org/2024.findings-acl.456/.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501v2*, 2025. URL https://arxiv.org/abs/2506.09501v2.

Yuan, T., Liao, C., and Wen, Z. Strong drafts need compact memories: Long-context speculative decoding with compressed KV cache. *arXiv preprint arXiv:2608.30252*, 2026. URL https://arxiv.org/abs/2608.30252. EMNLP 2026 Findings.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025. URL https://arxiv.org/abs/2412.18934v2.

Zhang, R. H., Dey, S., Mishra, A., Wu, H., Li, B., and Zhang, R. Correctness forensics for batch speculative decoding: Diagnosing the ragged tensor problem. *arXiv preprint arXiv:2510.22876v4*, 2026. URL https://arxiv.org/abs/2510.22876v4. Accepted to Findings of EMNLP 2026.